

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

THIPPIREDDYGARI SHIRDIJA(1BF24CS320)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by THIPPIREDDYGARI SHIRDIJA(1BF24CS320), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.

The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	4-10
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	11-18
3.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	19-22
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: Rate- Monotonic Earliest-deadline First Proportional scheduling	22-30
5.	Write a C program to simulate producer-consumer problem using semaphores	30-33
6.	Write a C program to simulate the concept of Dining Philosophers problem.	34-36
7.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	36-39

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include<stdio.h>
```

```
void sort(int proc_id[],int at[],int bt[],int n)
{
    int min=at[0],temp=0;
    for(int i=0;i<n;i++)
    {
        min=at[i];
        for(int j=i;j<n;j++)
        {
            if(at[j]<min)
            {
                temp=at[i];
                at[i]=at[j];
                at[j]=temp;
                temp=bt[j];
                bt[j]=bt[i];
                bt[i]=temp;
                temp=proc_id[i];
                proc_id[i]=proc_id[j];
                proc_id[j]=temp;
            }
        }
    }
}
```

```
void main()
{
    int n,c=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int proc_id[n],at[n],bt[n],ct[n],tat[n],wt[n];
```

```

double avg_tat=0.0,ttat=0.0,avg_wt=0.0,twt=0.0;
for(int i=0;i<n;i++)
    proc_id[i]=i+1;
printf("Enter arrival times:\n");
for(int i=0;i<n;i++)
    scanf("%d",&at[i]);
printf("Enter burst times:\n");
for(int i=0;i<n;i++)
    scanf("%d",&bt[i]);

sort(proc_id,at,bt,n);
//completion time
for(int i=0;i<n;i++)
{
    if(c>=at[i])
        c+=bt[i];
    else
        c+=at[i]-ct[i-1]+bt[i];
    ct[i]=c;
}
//turnaround time
for(int i=0;i<n;i++)
    tat[i]=ct[i]-at[i];
//waiting time
for(int i=0;i<n;i++)
    wt[i]=tat[i]-bt[i];

printf("FCFS scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",proc_id[i],at[i],bt[i],ct[i],tat[i],wt[i]);

for(int i=0;i<n;i++)
{
    ttat+=tat[i];twt+=wt[i];
}
avg_tat=ttat/(double)n;
avg_wt=twt/(double)n;
printf("\nAverage turnaround time:%lfms\n",avg_tat);
printf("\nAverage waiting time:%lfms\n",avg_wt);
}

```

Result:

1BF24CS320

```
enter no.of process4
enter arrival time0
enter brust time2
enter arrival time1
enter brust time2
enter arrival time5
enter brust time3
enter arrival time6
enter brust time4
```

P	AT	BT	CT	TAT	WT
P0	0	2	2	2	0
P1	1	2	4	3	1
P2	5	3	8	3	0
P3	6	4	12	6	2

Average TAT = 3.50

Average WT = 0.75

=>**SJF(Non-preemptive):**

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j,temp;
```

```
    printf("1BF24CS320\n\n");
```

```
    printf("enter no.of process");
```

```
    scanf("%d",&n);
```

```
    int at[n],bt[n],ct[n],tat[n],wt[n],p[n];
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        p[i]=i;
```

```
        printf("enter arrival time");
```

```
        scanf("%d",&at[i]);
```

```
        printf("enter brust time");
```

```
        scanf("%d",&bt[i]);
```

```
    }
```

```
    // Sorting according to Burst Time
```

```
    for(i=0;i<n-1;i++)
```

```

{
    for(j=i+1;j<n;j++)
    {
        if(bt[i] > bt[j])
        {
            temp=bt[i];
            bt[i]=bt[j];
            bt[j]=temp;
            temp=at[i];
            at[i]=at[j];
            at[j]=temp;
            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
        }
    }
}

```

```

ct[0]=at[0]+bt[0];
tat[0]=ct[0]-at[0];
wt[0]=tat[0]-bt[0];
for(i=1;i<n;i++)
{
    if(ct[i-1] < at[i])
        ct[i]=at[i]+bt[i];
    else
        ct[i]=ct[i-1]+bt[i];
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];
}
float avgtat=0, avgwt=0;
printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");

```

```

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    avgtat += tat[i];
    avgwt += wt[i];
}
avgtat = avgtat/n;
avgwt = avgwt/n;
printf("\nAverage TAT = %.2f",avgtat);
printf("\nAverage WT = %.2f",avgwt);
return 0;
}

```

Result:

```

1BF24CS320

enter no.of process4
enter arrival time0
enter brust time6
enter arrival time1
enter brust time8
enter arrival time2
enter brust time7
enter arrival time3
enter brust time3

P      AT      BT      CT      TAT      WT
P3      3      3      6      3      0
P0      0      6      12     12      6
P2      2      7      19     17     10
P1      1      8      27     26     18

Average TAT = 14.50
Average WT = 8.50

```

SJF (preemptive):

```
#include <stdio.h>
```

```
#include <limits.h>
```

```

int main() {
    int n;

    printf("Enter the number of processes: ");
    scanf("%d",&n);

```



```

int pid[n], at[n], bt[n], rt[n];
int ct[n], tat[n], wt[n];
for(int i=0;i<n;i++){
    pid[i] = i+1;
}
for(int i=0;i<n;i++){
    printf("Enter arrival time and burst time for process %d: ",i+1);
    scanf("%d %d",&at[i],&bt[i]);
    rt[i] = bt[i];
}
int completed = 0;
int t = 0;
while(completed < n){
    int idx = -1;
    int min_rt = INT_MAX;
    for(int i=0;i<n;i++){
        if(at[i] <= t && rt[i] > 0){
            if(rt[i] < min_rt ||
                (rt[i] == min_rt && at[i] < at[idx])){
                min_rt = rt[i];
                idx = i;
            } } }
    if(idx == -1){
        t++;
        continue;
    }
    rt[idx]--;
    t++;
    if(rt[idx] == 0){
        completed++;
        ct[idx] = t;
        tat[idx] = ct[idx] - at[idx];
    }
}

```

```

        wt[idx] = tat[idx] - bt[idx];
        if(wt[idx] < 0)
            wt[idx] = 0;
    }
}

float avg_wt = 0, avg_tat = 0;
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++){
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time: %.2f",avg_wt/n);
printf("\nAverage Turnaround Time: %.2f\n",avg_tat/n);
return 0;
}

```

Result:

1BF24CS320

```

enter no.of process4
enter arrival time0
enter brust time8
enter arrival time1
enter brust time4
enter arrival time2
enter brust time2
enter arrival time3
enter brust time1

```

P	AT	BT	CT	TAT	WT
P0	0	8	15	15	7
P1	1	4	8	7	3
P2	2	2	5	3	1
P3	3	1	4	1	0

```

Average TAT = 6.50
Average WT = 2.75

```

Program-2

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

→ Priority (pre-emptive)

→ Priority (pre-emptive)

Code:

```
#include <stdio.h>

int main()
{
    printf("shirdija");
    int n, i, current_time = 0, completed_count = 0;
    int pid[20], at[20], bt[20], pr[20];
    int ct[20], tat[20], wt[20], rt[20];
    int completed[20] = {0};
    float avg_tat = 0, avg_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i + 1);
        printf("Enter Process ID: ");
        scanf("%d", &pid[i]);
        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Enter Priority: ");
        scanf("%d", &pr[i]);
        rt[i] = bt[i];
    }

    while(completed_count < n)
    {
        int highest_priority = 9999;
        int selected = -1;
```

```

for(i = 0; i < n; i++)
{
    if(at[i] <= current_time && rt[i] > 0)
    {
        if(pr[i] < highest_priority)
        {
            highest_priority = pr[i];
            selected = i;
        }
    }
}

if(selected == -1)
{
    current_time++;
}
else
{
    rt[selected] = rt[selected] - 1;
    current_time += 1;
    if(rt[selected] == 0)
    {
        ct[selected] = current_time;
        tat[selected] = ct[selected] - at[selected];
        wt[selected] = tat[selected] - bt[selected];
        completed[selected] = 1;
        completed_count++;
    }
}
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage Turnaround Time = %.2f", avg_tat);

```

```

printf("\nAverage Waiting Time = %.2f\n", avg_wt);

return 0;
}

```

Result:

```

shirdijaEnter number of processes: 5

Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 5

Process 2
Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 2
Enter Priority: 3

Process 3
Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 5
Enter Priority: 2

Process 4
Enter Process ID: 4
Enter Arrival Time: 4
Enter Burst Time: 4
Enter Priority: 4

Process 5
Enter Process ID: 5
Enter Arrival Time: 6
Enter Burst Time: 1

Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 5
Enter Priority: 2

Process 4
Enter Process ID: 4
Enter Arrival Time: 4
Enter Burst Time: 4
Enter Priority: 4

Process 5
Enter Process ID: 5
Enter Arrival Time: 6
Enter Burst Time: 1
Enter Priority: 1

PID      AT      BT      PR      CT      TAT      WT
1         0        3        5        3        3        0
2         2        2        3       11        9        7
3         3        5        2        8        5        0
4         4        4        4       15       11        7
5         6        1        1        9        3        2

Average Turnaround Time = 6.20
Average Waiting Time = 3.20

```

→ Priority (Non-pre-emptive)

Code:

```
#include <stdio.h>

int main()
{
    printf("shirdija");
    int n, i, current_time = 0, completed_count = 0;
    int pid[20], at[20], bt[20], pr[20];
    int ct[20], tat[20], wt[20];
    int completed[20] = {0};
    float avg_tat = 0, avg_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i + 1);
        printf("Enter Process ID: ");
        scanf("%d", &pid[i]);
        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Enter Priority: ");
        scanf("%d", &pr[i]);
    }
    while(completed_count < n)
    {
        int highest_priority = 9999;
        int selected = -1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= current_time && completed[i] == 0)
            {
                if(pr[i] < highest_priority)
                {
                    highest_priority = pr[i];
                    selected = i;
                }
            }
        }
        if(selected == -1)
        {

```

```

        current_time++;
    }
    else
    {
        ct[selected] = current_time + bt[selected];
        tat[selected] = ct[selected] - at[selected];
        wt[selected] = tat[selected] - bt[selected];

        current_time = ct[selected];
        completed[selected] = 1;
        completed_count++;
    }
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f\n", avg_wt);

return 0;
}

```

Result:

```
"C:\Users\BMSCECSE-L4\Desktop" X + v
shirdijaEnter number of processes: 7

Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 8
Enter Priority: 3

Process 2
Enter Process ID: 2
Enter Arrival Time: 1
Enter Burst Time: 2
Enter Priority: 4

Process 3
Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 4
Enter Priority: 4

Process 4
Enter Process ID: 4
Enter Arrival Time: 4
Enter Burst Time: 1
Enter Priority: 5

Process 5
Enter Process ID: 5
Enter Arrival Time: 5
Enter Burst Time: 6
Enter Priority: 2

Process 6
Enter Process ID: 6
Enter Arrival Time: 6
Enter Burst Time: 5
Enter Priority: 6

Process 7
Enter Process ID: 7
Enter Arrival Time: 10
Enter Burst Time: 1
Enter Priority: 1

PID    AT    BT    PR    CT    TAT    WT
1      0      8      3     15     15     7
2      1      2      4     17     16     14
3      3      4      4     21     18     14
4      4      1      5     22     18     17
5      5      6      2     12      7      1
6      6      5      6     27     21     16
```

→Round Robin:

Code:

```
#include <stdio.h>
int main()
{
    printf("shirdija");
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &tq);
    int pid[n], at[n], bt[n], rt[n];
    int wt[n], tat[n], ct[n];
    int queue[100], front = 0, rear = 0;
    int visited[n];
    for(int i = 0; i < n; i++)
    {
        printf("\nEnter Process ID: ");
        scanf("%d", &pid[i]);
        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
```



```

    visited[i] = 0;
}
int current_time = 0;
int completed = 0;
double avg_wt = 0, avg_tat = 0;
while(completed < n)
{
    for(int i = 0; i < n; i++)
    {
        if(at[i] <= current_time && visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    if(front == rear)
    {
        current_time++;
        continue;
    }
    int p = queue[front++];
    if(rt[p] > tq)
    {
        current_time += tq;
        rt[p] -= tq;
        for(int i = 0; i < n; i++)
        {
            if(at[i] <= current_time && visited[i] == 0)
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
        queue[rear++] = p;
    }
    else
    {
        current_time += rt[p];
        rt[p] = 0;
        ct[p] = current_time;
        tat[p] = ct[p] - at[p];
        wt[p] = tat[p] - bt[p];
        avg_wt += wt[p];
        avg_tat += tat[p];
        completed++;
    }
}

```

```

    }
    avg_wt /= n;
    avg_tat /= n;
    for(int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2lf", avg_wt);
    printf("\nAverage Turnaround Time = %.2lf", avg_tat);
    return 0;
}

```

Result:

```

shirdijaEnter number of processes: 5
Enter time quantum: 2

Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 5

Enter Process ID: 2
Enter Arrival Time: 1
Enter Burst Time: 3

Enter Process ID: 3
Enter Arrival Time: 2
Enter Burst Time: 1

Enter Process ID: 4
Enter Arrival Time: 3
Enter Burst Time: 2

Enter Process ID: 5
Enter Arrival Time: 4
Enter Burst Time: 3
1      0      5      13      8      13
2      1      3      12      8      11
3      2      1      5       2      3
4      3      2      9       4      6
5      4      3      14      7      10

Average Waiting Time = 5.80
Average Turnaround Time = 8.60

```

Program-3:

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue

Code:

```
#include <stdio.h>

#define MAX 50

typedef struct {
    int pid, at, bt, rt, ct, tat, wt, qno;
} Process;

int main() {
    printf("shirdija");
    Process p[MAX];
    int n, i, time = 0, completed = 0;
    int tq;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum for Queue 1 (RR): ");
    scanf("%d", &tq);

    for (i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);

        printf("PID: ");
        scanf("%d", &p[i].pid);

        printf("Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Burst Time: ");
        scanf("%d", &p[i].bt);

        printf("Queue Number (1=RR, 2=FCFS): ");
        scanf("%d", &p[i].qno);

        p[i].rt = p[i].bt;
    }
```

```

while (completed < n) {

    int executed = 0;

    // ----- Queue 1 (Round Robin) -----
    for (i = 0; i < n; i++) {
        if (p[i].qno == 1 && p[i].at <= time && p[i].rt > 0) {

            executed = 1;

            int run = (p[i].rt > tq) ? tq : p[i].rt;

            time += run;
            p[i].rt -= run;

            if (p[i].rt == 0) {
                p[i].ct = time;
                completed++;
            }
        }
    }

    // ----- Queue 2 (Preemptible FCFS) -----
    if (!executed) {
        for (i = 0; i < n; i++) {
            if (p[i].qno == 2 && p[i].at <= time && p[i].rt > 0) {

                executed = 1;

                // Execute 1 unit only → allows preemption
                time++;
                p[i].rt--;

                if (p[i].rt == 0) {
                    p[i].ct = time;
                    completed++;
                }

                break;
            }
        }
    }

    // CPU Idle
    if (!executed) {
        time++;
    }
}

```

```

    }
}

// ----- Calculate TAT & WT -----
float total_tat = 0, total_wt = 0;

printf("\nPID\tAT\tBT\tQ\tCT\tTAT\tWT\n");

for (i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;

    total_tat += p[i].tat;
    total_wt += p[i].wt;

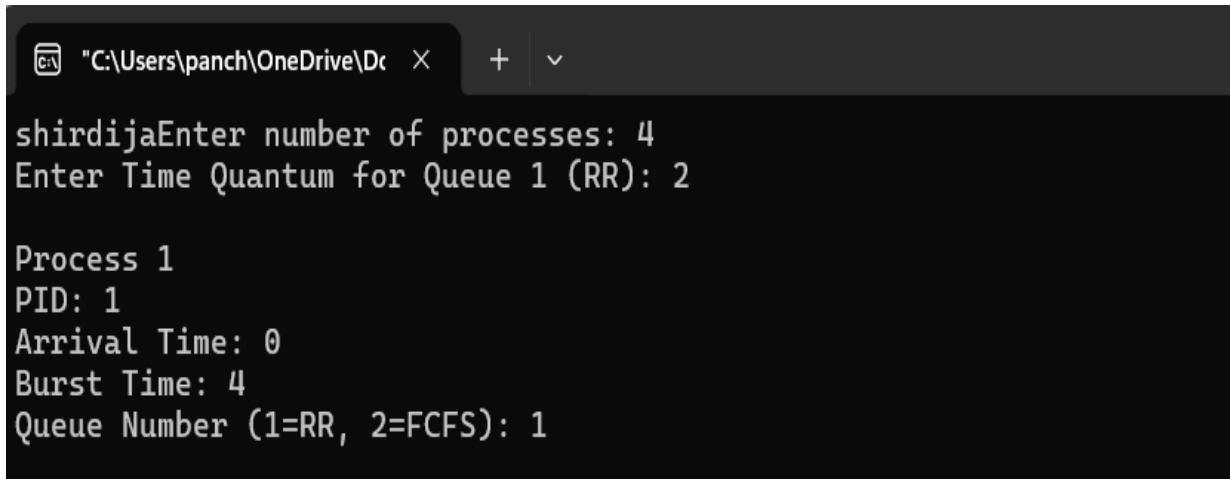
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].qno,
        p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage Turnaround Time = %.2f", total_tat / n);
printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

return 0;
}

```

Result:



```

C:\Users\panch\OneDrive\Dr... × + v
shirdijaEnter number of processes: 4
Enter Time Quantum for Queue 1 (RR): 2

Process 1
PID: 1
Arrival Time: 0
Burst Time: 4
Queue Number (1=RR, 2=FCFS): 1

```

```
"C:\Users\panch\OneDrive\De  x + v
Process 2
PID: 2
Arrival Time: 0
Burst Time: 3
Queue Number (1=RR, 2=FCFS): 1

Process 3
PID: 3
Arrival Time: 0
Burst Time: 8
Queue Number (1=RR, 2=FCFS): 2

Process 4
PID: 4
Arrival Time: 10
Burst Time: 5
Queue Number (1=RR, 2=FCFS): 1

PID    AT    BT    Q    CT    TAT    WT
P1     0     4     1     6     6     2
P2     0     3     1     7     7     4
P3     0     8     2    20    20    12
P4    10     5     1    15     5     0

Average Turnaround Time = 9.50
Average Waiting Time = 4.50

Process returned 0 (0x0)   execution time : 55.995 s
Press any key to continue.
```

Program-4:

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a)Rate- Monotonic
- b)Earliest-deadline First
- c)Proportional scheduling
- a)Rate- Monotonic

Code:

```
#include <stdio.h>
#include <math.h>

#define MAX 10

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
```

```

}

int find_lcm(int arr[], int n) {
    int res = arr[0];
    for (int i = 1; i < n; i++) {
        res = lcm(res, arr[i]);
    }
    return res;
}

int main() {
    printf("shirdija");
    int n, i, j;
    int C[MAX], P[MAX], remaining[MAX];

    printf("Enter the number of processes:");
    scanf("%d", &n);

    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &C[i]);

    printf("Enter the time periods:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &P[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (P[i] > P[j]) {
                int temp;
                temp = P[i]; P[i] = P[j]; P[j] = temp;
                temp = C[i]; C[i] = C[j]; C[j] = temp;
            }
        }
    }

    int hyper = find_lcm(P, n);
    printf("LCM=%d\n\n", hyper);

    printf("Rate Monotone Scheduling:\n");
    printf("PID\tBurst\tPeriod\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\n", i + 1, C[i], P[i]);
    }
}

```

```

double U = 0;
for (i = 0; i < n; i++) {
    U += (double)C[i] / P[i];
}

double bound = n * (pow(2, (double)1/n) - 1);

printf("\n%f <= %f => %s\n",
    U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", hyper);

// Initialize
for (i = 0; i < n; i++)
    remaining[i] = 0;

for (int t = 0; t < hyper; t++) {

    for (i = 0; i < n; i++) {
        if (t % P[i] == 0) {
            remaining[i] = C[i];
        }
    }

    int executed = -1;

    for (i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            executed = i;
            break;
        }
    }

    if (executed != -1) {
        printf("%dms onwards: Process %d running\n", t, executed + 1);
        remaining[executed]--;
    } else {
        printf("%dms onwards: CPU is idle\n", t);
    }
}

return 0;

```


}

Result:

```
"C:\Users\BMSCECSE-L4\Desk  X + v
shirdijaEnter the number of processes:3
Enter the CPU burst times:
3 2 1
Enter the time periods:
20 5 10
LCM=20
```

Rate Monotone Scheduling:

PID	Burst	Period
1	2	5
2	1	10
3	3	20

```
0.650000 <= 0.779763 => true
Scheduling occurs for 20 ms
```

```
0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 2 running
3ms onwards: Process 3 running
4ms onwards: Process 3 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 3 running
8ms onwards: CPU is idle
9ms onwards: CPU is idle
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 2 running
13ms onwards: CPU is idle
14ms onwards: CPU is idle
15ms onwards: Process 1 running
16ms onwards: Process 1 running
17ms onwards: CPU is idle
18ms onwards: CPU is idle
19ms onwards: CPU is idle
```

```
Process returned 0 (0x0)  execution time : 53.086 s
Press any key to continue.
```

Earliest-deadline First:

Code:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 999999
```

```
int gcd(int a, int b) {  
    if (b == 0) return a;  
    return gcd(b, a % b);  
}
```

```
int lcm(int a, int b) {  
    return (a * b) / gcd(a, b);  
}
```

```
int find_lcm(int arr[], int n) {  
    int res = arr[0];  
    for (int i = 1; i < n; i++) {  
        res = lcm(res, arr[i]);  
    }  
    return res;  
}
```

```
int main() {  
    int n, i;  
    printf("shirdija");
```

```
  
    int burst[MAX], deadline[MAX], period[MAX];  
    int remaining[MAX];  
    int abs_deadline[MAX];
```

```
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);
```

```
  
    printf("Enter Burst Time, Deadline, Period:\n");  
    for (i = 0; i < n; i++) {  
        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);  
        remaining[i] = 0;  
        abs_deadline[i] = INF;  
    }
```

```
  
    int hyper = find_lcm(period, n);
```

```

printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
}

printf("Scheduling occurs for %d ms\n\n", hyper);

for (int time = 0; time < hyper; time++) {

    for (i = 0; i < n; i++) {
        if (time % period[i] == 0) {
            remaining[i] = burst[i];
            abs_deadline[i] = time + deadline[i];
        }
    }

    int selected = -1;
    int min_deadline = INF;

    for (i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (abs_deadline[i] < min_deadline) {
                min_deadline = abs_deadline[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("%dms: CPU is idle.\n", time);
    } else {
        printf("%dms: Task %d is running.\n", time, selected + 1);

        remaining[selected]--;

        if (remaining[selected] == 0) {
            abs_deadline[selected] = INF;
        }
    }
}

```

```

    return 0;
}

```

Result:

```

6 int gcd(int a, int b) {
    return 0;
}

shirdijaEnter number of processes: 3
Enter Burst Time, Deadline, Period:
3 7 20
2 4 5
2 8 10

PID    Burst    Deadline    Period
1       3         7           20
2       2         4           5
3       2         8           10
Scheduling occurs for 20 ms

0ms: Task 2 is running.
1ms: Task 2 is running.
2ms: Task 1 is running.
3ms: Task 1 is running.
4ms: Task 1 is running.
5ms: Task 3 is running.
6ms: Task 3 is running.
7ms: Task 2 is running.
8ms: Task 2 is running.
9ms: CPU is idle.
10ms: Task 2 is running.
11ms: Task 2 is running.
12ms: Task 3 is running.
13ms: Task 3 is running.
14ms: CPU is idle.
15ms: Task 2 is running.
16ms: Task 2 is running.
17ms: CPU is idle.
18ms: CPU is idle.
19ms: CPU is idle.

```

Proportional scheduling:

Code:

```

#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
    char name[5]; int tickets; }
Process;
int main() {
    int n, total_tickets = 0; float total_T =0.0;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));

```

```

for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
  sprintf(p[i].name, "P%d", i + 1);
  printf("Tickets: ");
  scanf("%d", &p[i].tickets);
  total_tickets += p[i].tickets;
  total_T += p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
  int winning_ticket = rand() % total_tickets + 1;
  int accumulated_tickets = 0;
  int winner_index;
  for (int j = 0; j < n; j++)
  {
    accumulated_tickets += p[j].tickets;
    if (winning_ticket <= accumulated_tickets)
    {
      winner_index = j; break;
    }
  }
  printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
    p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
  printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
    ((p[i].tickets / total_T) * 100));
}
return 0;
}

```

Result:

```

Enter the number of Processes: 3
Process 1:
Tickets: 10
Process 2:
Tickets: 20
Process 3:
Tickets: 30
--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 3, Winner: P1
Tickets picked: 48, Winner: P3
Tickets picked: 33, Winner: P3
Tickets picked: 48, Winner: P3
Tickets picked: 55, Winner: P3
1BF24CS320

The Process: P1 gets 16.67% of Processor Time.
The Process: P2 gets 33.33% of Processor Time.
The Process: P3 gets 50.00% of Processor Time.
Process returned 0 (0x0)   execution time : 35.387 s
Press any key to continue.

```

Program-5:

Question:

Write a C program to simulate producer-consumer problem using semaphores

Code:

```

#include <stdio.h>
#include <stdlib.h>

```

```

#define SIZE 5

```

```

int stack[SIZE];
int top = -1;

```

```

// Semaphore variables
int empty = SIZE;
int full = 0;
int mutex = 1;

```

```

// WAIT operation
void wait(int *s) {
    (*s)--;
}

```

```
}
```

```
void signal(int *s) {  
    (*s)++;  
}
```

```
void producer() {  
    int item;  
  
    if (empty == 0) {  
        printf("\nBuffer is FULL! Producer cannot produce.\n");  
        return;  
    }  
  
    printf("\n[ENTRY] Enter item to produce: ");  
    scanf("%d", &item);  
  
    wait(&empty);  
    wait(&mutex);  
  
    // CRITICAL SECTION (push into stack)  
    top++;  
    stack[top] = item;  
    printf("[CRITICAL] Produced item %d at position %d\n", item, top);  
  
    signal(&mutex);  
    signal(&full);  
  
    printf("[EXIT] Producer done\n");  
}
```

```
void consumer() {  
    int item;  
  
    if (full == 0) {  
        printf("\nBuffer is EMPTY! Consumer cannot consume.\n");  
        return;  
    }  
  
    wait(&full);  
    wait(&mutex);  
  
    item = stack[top];  
    printf("[CRITICAL] Consumed item %d from position %d\n", item, top);  
    top--;
```

```
signal(&mutex);
signal(&empty);
```

```
printf("[EXIT] Consumer done\n");
}
```

```
void display() {
    int i;
    if (top == -1) {
        printf("\nStack is empty\n");
        return;
    }
```

```
    printf("\nStack contents: ");
    for (i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}
```

```
int main() {
    int choice;
```

```
    while (1) {
        printf("\n--- Producer Consumer (Stack) ---\n");
        printf("1. Produce\n2. Consume\n3. Display\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
```

```
        switch (choice) {
            case 1:
                producer();
                break;
            case 2:
                consumer();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice\n");
```


return 0;

}

Result:

```
--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 1

[ENTRY] Enter item to produce: 10
[CRITICAL] Produced item 10 at position 0
[EXIT] Producer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 1

[ENTRY] Enter item to produce: 20
[CRITICAL] Produced item 20 at position 1
[EXIT] Producer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 3

Stack contents: 20 10

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 2
[CRITICAL] Consumed item 20 from position 1
[EXIT] Consumer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 3

Stack contents: 10

--- Producer Consumer (Stack) ---
```

```
--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 3

Stack contents: 10

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 2
[CRITICAL] Consumed item 10 from position 0
[EXIT] Consumer done

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 2

Buffer is EMPTY! Consumer cannot consume.

--- Producer Consumer (Stack) ---
1. Produce
2. Consume
3. Display
4. Exit
Enter choice: 4

Process returned 0 (0x0)   execution time : 1611.640 s
Press any key to continue.
```

Program-6:

Question:

Write a C program to simulate the concept of Dining Philosophers problem.

Code:

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5 // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id; // left fork index
    int right = (id + 1) % N; // right fork index
    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking
        // To avoid deadlock, philosophers with even id pick left then right,
        // odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
            // Pick up right fork

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); }
        else
        { // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
            fork pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        // Eating
        printf("Philosopher %d is eating.\n", id);
        sleep(2); // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
```

```

}
return NULL;
}
int main() {
int i;
int ids[N]; // Initialize forks (mutexes)
printf("1BF24CS317");
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
// Create philosopher threads
for (i = 0; i < N; i++)

{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
} // Join threads (will never happen here, but good practice)
for (i = 0; i < N; i++)
{
pthread_join(philosophers[i], NULL); }
// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}
Result:

```

F:\os\multilevel.exe

```
1BF24CS320
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
```

Program-7:

Question:

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

Code:

```
#include <stdio.h>
```

```
int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
```

```

}

printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n];
int work[m];
for(i = 0; i < m; i++)
    work[i] = avail[i];
for(i = 0; i < n; i++)
    finish[i] = 0;
int count = 0;
while(count < n) {
    int found = 0;
    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            int possible = 1;
            for(j = 0; j < m; j++) {
                if(need[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }
            if(possible) {

                for(k = 0; k < m; k++) {
                    work[k] += alloc[i][k];
                }

                safeSeq[count] = i;
                count++;
            }
        }
    }
}

```

```

        finish[i] = 1;
        found = 1;
    }
}

if(found == 0) {
    break;
}

if(count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("1bf24cs338\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)
            printf(" -> ");
    }

    printf("\n");
}
else {
    printf("\nSystem is NOT in safe state.\n");
}

return 0;
}

```

Result:

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3
3
Enter Available Resources:
3 3 2

System is in a safe state.
1bf24cs320
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 49.227 s
Press any key to continue.
```